Métodos ágeis e gestão de projetos

Scrum, XP, Kanban, modelos de processo e áreas de conhecimento do PMBOK.

A questão testa conhecimentos a respeito de métodos ágeis. Em especial, precisamos conhecer os métodos Scrum, eXtreme Programming e Kanban. Estudaremos esses tópicos na sequência.

Métodos ágeis

Um processo de desenvolvimento de software define um conjunto de passos, tarefas, eventos e práticas que devem ser seguidos por desenvolvedores na elaboração de um sistema. A adoção de um processo específico é particularmente importante em projetos atuais, pois os sistemas de software modernos são, majoritariamente, muito complexos para serem desenvolvidos por um único programador. Nesse caso, a equipe de desenvolvimento precisa seguir diretrizes para produzir softwares com qualidade e com produtividade. Essas diretrizes são estabelecidas pelo processo de desenvolvimento adotado.

Os primeiros processos de desenvolvimento de software, propostos na década de 1970, eram estritamente sequenciais, seguindo o modelo em cascata. Os projetos eram iniciados com a especificação de requisitos e avançavam até chegar às fases de implementação, de testes e de manutenção. A figura 1 ilustra um processo de desenvolvimento de software seguindo modelo cascata, no qual o sistema é implementado e testado apenas nas etapas finais.

Figura 1. Desenvolvimento de software usando um processo baseado em modelo cascata.

VALENTE, [s.d.] M. T. Engenharia de software moderna. Disponível em <https://engsoftmoderna.info/>. Acesso em 15 dez. 2023.

Com o passar dos anos, profissionais da indústria de software propuseram alternativas aos processos em modelo em cascata. Eles afirmaram que um produto de software é diferente de produtos tradicionais de engenharia e, por isso, os softwares demandam um processo de desenvolvimento diferente.

Os requisitos de um software mudam com frequência, mais do que os requisitos de um computador ou de uma ponte, por exemplo. Adicionalmente, os clientes, muitas vezes, não têm uma ideia precisa do que querem, ou não conseguem expressar seus anseios adequadamente. Com isso, há o risco de se projetar por um longo tempo um produto de software que, depois de pronto, não será mais necessário, ou porque o mundo mudou, ou porque as necessidades dos clientes mudaram.

Nesse cenário, uma das alternativas propostas pelos profissionais da indústria foi um novo modelo de processo de software, que são os métodos ágeis. Os métodos ágeis são processos de desenvolvimento de software baseados em ciclos iterativos e incrementais de trabalho, nos quais são incentivadas a colaboração e a comunicação entre todas as pessoas envolvidas no projeto. Desse modo, um sistema de software pode ser implementado de maneira gradativa, começando pelas partes mais urgentes para o cliente.

De forma geral, podemos imaginar que, inicialmente, a equipe implementa uma primeira versão do sistema, contendo as funcionalidades prioritárias. Em sequência, essa versão é validada pelo cliente e, caso seja aprovada, um novo ciclo de implementação se inicia. A segunda versão do sistema, portanto, engloba mais funcionalidades do que a primeira e também deve ser validada pelo cliente. O processo continua iterativamente, de forma incremental, até o cliente atestar que todos os requisitos foram atendidos. Etapas do processo podem ser realizadas em paralelo, e os

processos ágeis costumam ser bem flexíveis em relação às suas etapas do processo de desenvolvimento.

A figura 2 ilustra um processo de desenvolvimento de software seguindo um método ágil, no qual o sistema é implementado, testado e validado em etapas incrementais. Na figura, cada iteração é representada por um retângulo e os incrementos no sistema são representados por S++. Observe que, a cada iteração, a equipe de desenvolvimento gera um incremento no sistema, que pode ser validado e testado pelos clientes antes da próxima iteração. Na prática, incrementos distintos podem ser desenvolvidos em paralelo, dependendo do tipo de projeto.

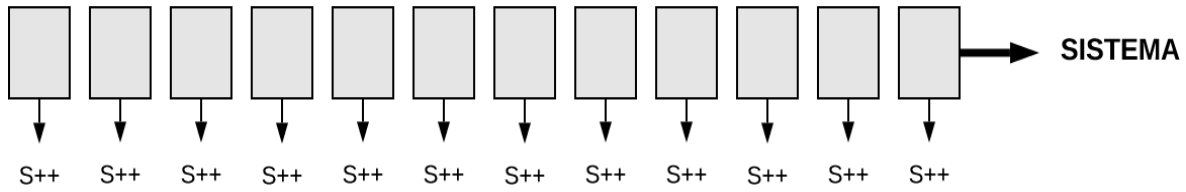


Figura 2. Desenvolvimento de software usando um método ágil.

VALENTE, [s.d.] M. T. Engenharia de software moderna. Disponível em <https://engsoftmoderna.info/>. Acesso em 15 dez. 2025.

Três dos principais processos de desenvolvimento de software que seguem a metodologia ágil são: Scrum, eXtreme Programming e Kanban. Estudaremos brevemente cada um deles, a seguir.

Scrum

O método Scrum, entre os métodos ágeis, é o mais conhecido e utilizado. Trata-se de um método para gerenciamento de projetos que, comumente, é aplicado a desenvolvimento de software, mas que não se restringe a essa aplicação. Como esse método pode ser aplicado a projetos em geral, o método não propõe qualquer prática de programação específica.

Na definição do Scrum, são estabelecidos papéis, eventos e artefatos. Veremos alguns detalhes a respeito desses três itens, a seguir.

Os papéis referem-se à especificação das pessoas envolvidas no projeto, com suas respectivas responsabilidades. São estabelecidos três papéis principais, elencados a seguir.

- Product Owner: tem o papel de representar os clientes. Para isso, o Product Owner deve entender do domínio do sistema que será construído, além de ser responsável por escrever os documentos que descrevem os requisitos do software, em linguagem natural.
- Scrum Master: é o especialista em Scrum do time, que é responsável por garantir que as regras do método estejam sendo seguidas.
- Desenvolvedores: trata-se de um grupo multifuncional, cujo número de integrantes varia entre 3 e 9. Os desenvolvedores são responsáveis por fazer a análise, a implementação, os testes e outras atividades pertinentes ao projeto.

Os eventos referem-se às atividades planejadas com o intuito de criar uma rotina de acompanhamento e de desenvolvimento do projeto. Os dois principais eventos são elencados a seguir.

- Sprints: assim como em todo método ágil, Scrum é um método iterativo. No Scrum, o nome dado para cada iteração é sprint. O método estabelece que cada sprint deve durar até um mês.
- Planejamento de sprints: é uma reunião na qual todo o time se reúne para decidir as histórias que serão implementadas no sprint que vai se iniciar. As histórias constituem uma descrição resumida das funcionalidades que devem ser implementadas no projeto, ou seja, estabelecem

os requisitos dos usuários.

Os artefatos dizem respeito aos documentos que os membros do projeto criam e utilizam ao longo do seu desenvolvimento. Há dois artefatos oficiais, elencados a seguir.

- Product backlog: trata-se de uma lista de histórias e de outros itens de trabalho relevantes, ordenada por prioridades. Conforme já abordamos, as histórias são uma descrição resumida das funcionalidades que devem ser implementadas no projeto. O product backlog é escrito pelo Product Owner.
- Sprint backlog: é o documento gerado ao final do evento de planejamento de sprint. Ele é constituído por uma lista de tarefas do sprint seguinte, com suas respectivas durações. Basicamente, o sprint backlog é uma lista de itens selecionados do product backlog, que devem ser implementados na próxima sprint.

eXtreme Programming

O método eXtreme Programming (XP) é um método ágil destinado ao desenvolvimento de softwares cujos requisitos sejam imprecisos ou estejam sujeitos a mudanças. Portanto, esse método, diferentemente do Scrum, é destinado especificamente para projetos de software.

O método envolve um conjunto de regras e práticas em suas iterações, que envolvem quatro atividades: planejamento, projeto, codificação e testes.

No planejamento, o representante do cliente cria as histórias de usuários, que descrevem o resultado, as características e a funcionalidade solicitados para o software. Cada história é colocada em uma ficha, e o cliente deve atribuir uma prioridade a cada uma delas. Os membros da equipe XP avaliam cada história e atribuem um custo (medido em semanas de desenvolvimento) a ela. A qualquer momento, podem ser escritas novas histórias.

O projeto XP estimula o uso de cartões CRC (classe-responsabilidade-colaborador) como uma ferramenta para pensar o software em um padrão orientado a objetos. Os cartões CRC identificam e organizam as classes relevantes para o incremento do software corrente. Os cartões CRC são, comumente, o único artefato de projeto produzido.

Após criar as histórias e elaborar o projeto, o time passará para a etapa de codificação. A equipe deve, primeiramente, desenvolver uma série de testes de unidades, que exercitarão as histórias que serão incluídas na versão atual do software. Uma vez criado o teste, os desenvolvedores podem se concentrar melhor no que deve ser implementado para que o software seja aprovado. Assim, com o código da versão atual do software completo, ele poderá ser testado imediatamente.

Na etapa de testes, a equipe precisa executar os testes de unidades, para corrigir eventuais falhas. Os testes de unidades devem ser implementados de forma automatizada pois, assim, poderão ser executados diversas vezes.

De acordo com o método XP, o desenvolvimento de projetos de software deve ser norteado por três valores principais, elencados a seguir.

- Comunicação: uma boa comunicação entre as partes envolvidas é importante em qualquer projeto, tanto para evitar que erros aconteçam quanto para aprender com os erros cometidos.
- Simplicidade: em todo sistema complexo, existem subsistemas mais simples, que devem ser considerados.
- Feedback: estar aberto ao feedback das partes envolvidas no projeto permite que correções de rota sejam implementadas o quanto antes, controlando os riscos aos quais o projeto está exposto.

Além dos valores, o XP é definido por um conjunto de princípios e de práticas de desenvolvimento, cujo extenso conteúdo não será abordado neste material.

Kanban

O método Kanban consiste em um processo visual, que utiliza cartões para descrever as histórias de usuário e os passos do processo de desenvolvimento do projeto. Quando comparamos o Kanban com o Scrum, percebemos que esse é um método mais simples, que não tem eventos, papéis pré-definidos ou diversos artefatos. O único artefato Kanban é o quadro de tarefas, que é chamado de Quadro Kanban. Esse quadro inclui, também, o product backlog.

O Quadro Kanban é dividido em colunas. A primeira coluna representa o próprio product backlog, com as histórias de usuários. As demais colunas são os passos que devem ser seguidos para transformar uma história em uma funcionalidade. Por exemplo, podemos criar as colunas Análise, Desenvolvimento e Testes. Além disso, cada coluna pode ser dividida em duas subcolunas: “tarefas em execução” e “tarefas concluídas”. Por exemplo, a coluna Desenvolvimento pode ter duas subcolunas: “tarefas em desenvolvimento” e “tarefas desenvolvidas”.

Portanto, a ideia do Kanban é que as histórias sejam processadas passo a passo, da esquerda para a direita, como em uma linha de montagem. Esse processo visual ajuda o time a identificar gargalos e a melhorar a eficiência do desenvolvimento do projeto. Podemos ver um exemplo de Quadro Kanban na figura 3, no qual duas colunas (Análise e Desenvolvimento) foram subdivididas.

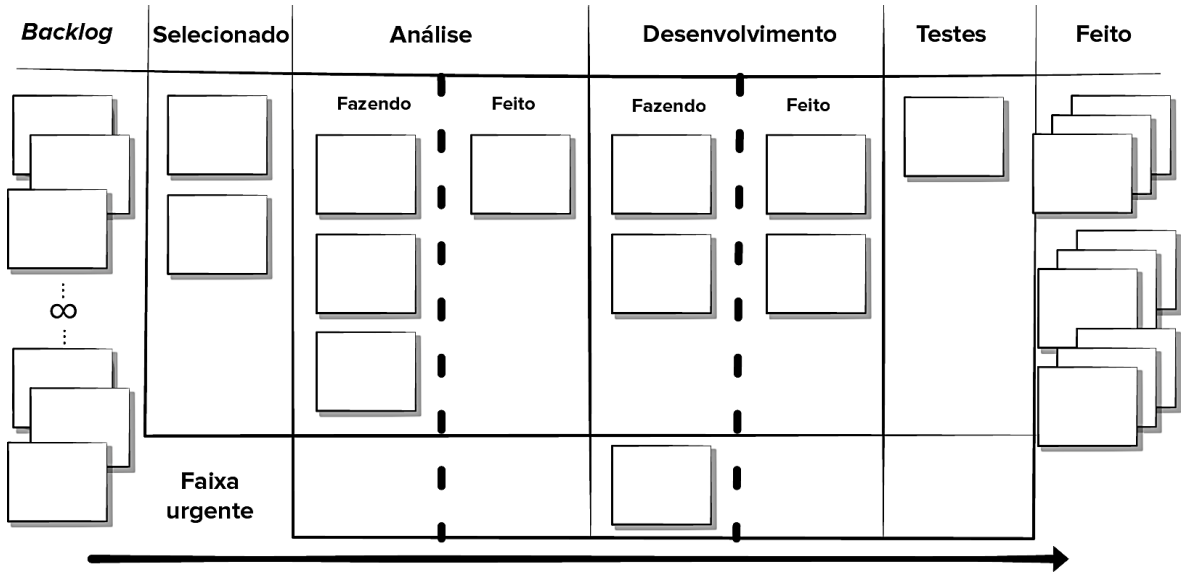


Figura 3. Exemplo de Quadro Kanban.

PRESSMAN e MAXIM, 2021.

A questão requer conhecimentos a respeito de tipos distintos de modelos de processos de software. Abordaremos, a seguir, alguns deles: cascata, espiral e incremental. Adicionalmente, faremos algumas comparações com o modelo ágil, já abordado na Introdução Teórica da questão 6.

Modelos de processos de software

Conforme vimos na Introdução teórica 6, um processo de software pode ser definido como uma sequência de atividades, tarefas, métodos e práticas que são realizadas durante o desenvolvimento de um sistema de software. Nesse contexto, um modelo de processos de

software é uma representação abstrata de como um processo de software deveria ser organizado e executado.

Um mesmo modelo pode dar origem a diferentes processos. Por exemplo, o modelo ágil representa um conjunto de abordagens colaborativas e iterativas, como Scrum, XP e Kanban.

Na prática, é possível combinar elementos de dois ou de diversos modelos de processo de software em um único processo personalizado, criando um modelo híbrido. Nesse caso, o objetivo é tirar proveito das vantagens de diferentes modelos para atender às necessidades específicas de um projeto ou de uma organização. Vamos estudar, a seguir, alguns dos modelos descritos na literatura de engenharia de software.

1.1.1. Modelo cascata

O modelo cascata prevê para o projeto um fluxo de trabalho previsível. Esse modelo é útil quando os requisitos do projeto são estáveis e foram bem compreendidos no início do projeto.

O modelo cascata traz uma abordagem linear e sistemática para o desenvolvimento do software, que começa com a especificação de requisitos de usuário (na etapa de comunicação) e avança para o planejamento, a modelagem, a construção e, finalmente, para a entrega. Na etapa de entrega, está previsto, também, o suporte ao usuário. No diagrama da figura 1, podemos ver as etapas previstas pelo modelo cascata.

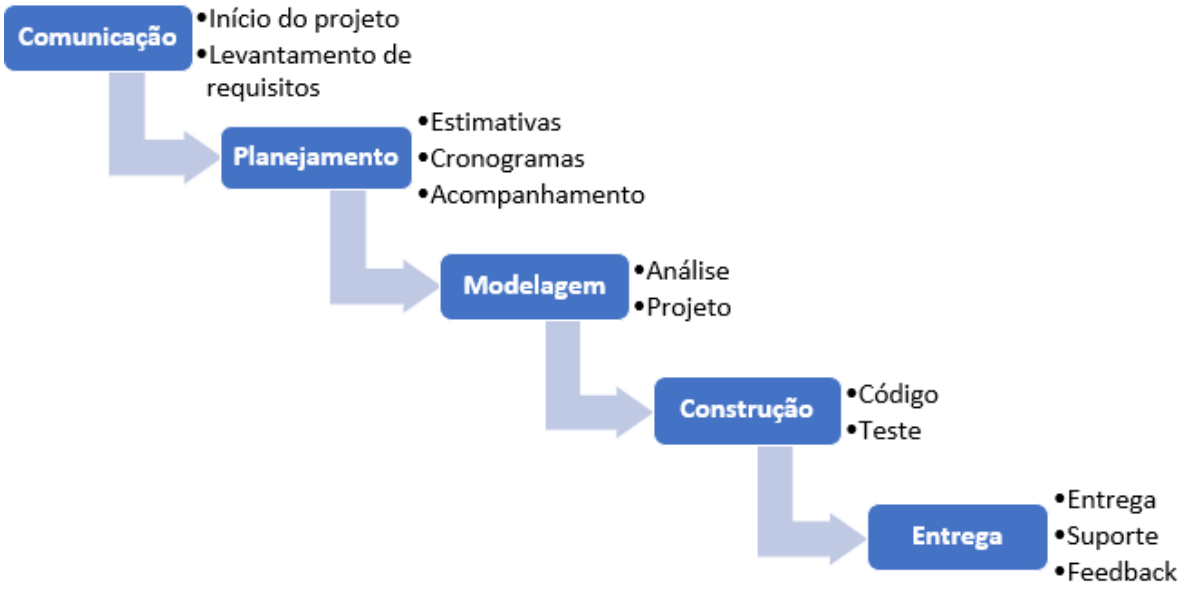


Figura 1. Modelo cascata.

PRESSMAN e MAXIM, 2021 (com adaptações).

1.1.2. Modelo espiral

O modelo espiral traz uma abordagem evolutiva ao desenvolvimento do software. Ele é adequado para casos em que um conjunto importante de requisitos é bem conhecido, mas precisa ser aprimorado ao longo da evolução do projeto. O modelo espiral é um modelo de processo de software que une o dinamismo da prototipação aos aspectos sistemáticos do modelo cascata. Processos que adotam esse modelo promovem o rápido desenvolvimento de versões cada vez mais completas do software desejado.

O modelo espiral é dividido em etapas, e cada uma constitui uma atividade definida pela equipe de engenharia de software. Para exemplificar, utilizaremos as etapas de comunicação, planejamento, modelagem, construção e entrega, conforme vemos na figura 2. Cada uma dessas atividades

representa um segmento do caminho espiral ilustrado. Assim que esse processo começa, a equipe de software realiza atividades indicadas por um circuito em torno da espiral, no sentido horário, começando pelo seu centro.

O primeiro circuito em volta da espiral (iniciando na região da linha de fluxo mais próxima ao centro) pode resultar no desenvolvimento de uma especificação de produto. Passagens posteriores em torno da espiral podem ser usadas para desenvolver um protótipo e, então, progressivamente, versões cada vez mais sofisticadas do software. Cada passagem pela região de planejamento resulta em ajustes no planejamento do projeto. Custo e cronograma são ajustados de acordo com o feedback do cliente após a entrega. Além disso, o gerente de projeto faz um ajuste no número de iterações planejadas para concluir o software.

O modelo espiral destaca a avaliação contínua dos resultados e as atividades de gerenciamento de riscos. Por isso, os riscos são levados em conta à medida que cada revolução é realizada. Ele usa a prototipação como mecanismo de redução de riscos. O modelo espiral, quando aplicado apropriadamente, reduz os riscos antes de eles se tornarem problemáticos.

Em comparação com o modelo ágil, o modelo espiral pode ser mais apropriado para projetos nos quais a gestão de riscos é crítica, enquanto o modelo ágil é mais adequado para projetos em que os requisitos estão sujeitos a mudanças frequentes.

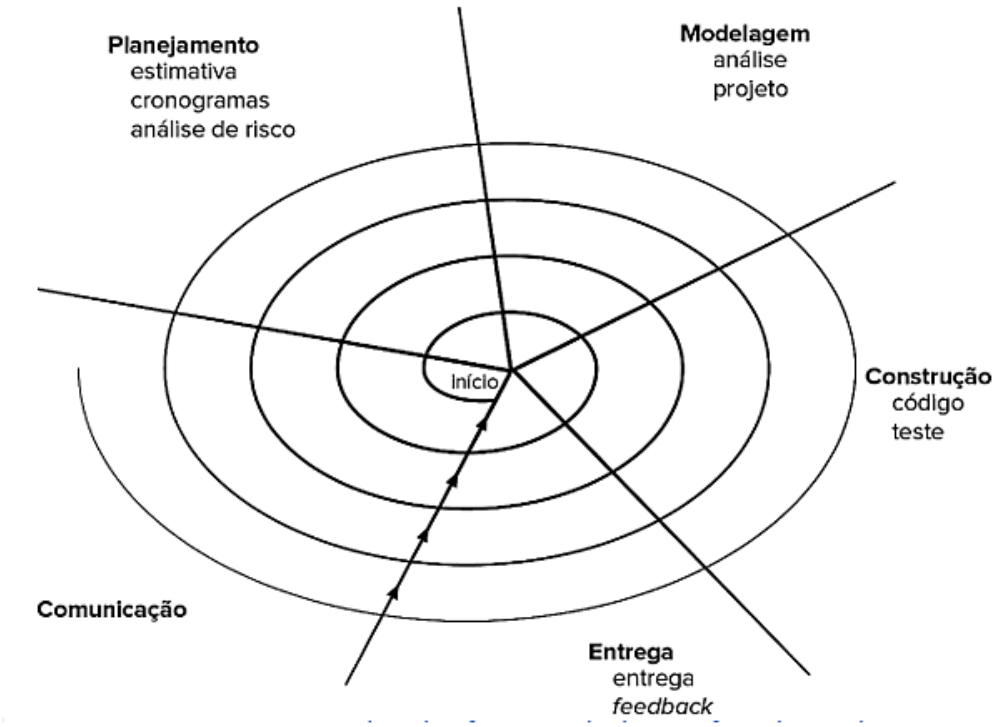


Figura 2. Modelo espiral.

PRESSMAN e MAXIM, 2021.

1.1.3. Modelo incremental

O modelo incremental traz uma abordagem evolutiva ao desenvolvimento do software. Geralmente, ele combina elementos dos fluxos de processos tanto lineares quanto paralelos. Cada uma das sequências lineares gera um incremento do software, que pode ser apresentado aos clientes.

O modelo incremental aplica sequências lineares de etapas (como no modelo cascata) de forma escalonada, à medida que o tempo avança. Essas sequências podem ter etapas paralelas entre si, conforme podemos ver na figura 3. Cada uma das sequências lineares gera um incremento do software.

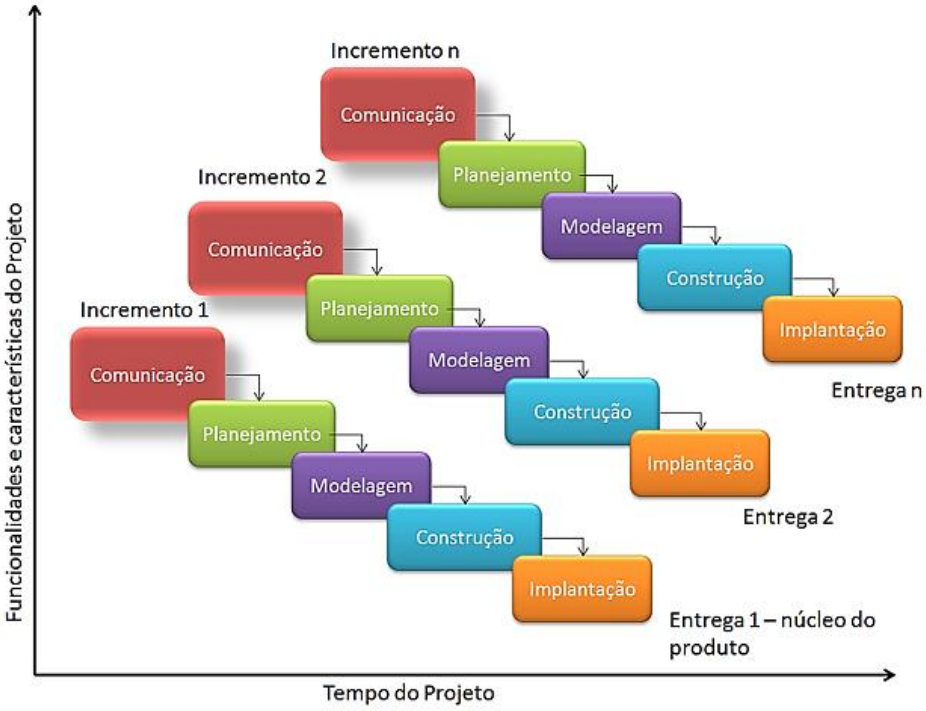


Figura 3. Modelo incremental.

Disponível em <http://mainedlystorm.weebly.com/blog/ciclo-de-vida-de-software-incremental>. Acesso em 19 jan. 2026.

Nesse modelo, é importante gerenciar adequadamente a coordenação entre as equipes e as diferentes partes do projeto, para garantir a integração dos incrementos. Além disso, é necessário considerar a comunicação eficiente e a resolução de possíveis conflitos que podem surgir quando várias partes do sistema estão sendo desenvolvidas simultaneamente, de forma paralela.

Apesar de poder ser considerado, por si, como um modelo de processo de software, o termo “incremental” pode ser utilizado para se referir a qualquer modelo que utilize incrementos em sua metodologia. Nesse sentido, o modelo ágil pode ser considerado como um dos modelos incrementais. Enquanto o modelo ágil é geralmente considerado incremental, nem todos os processos baseados em modelo incremental são necessariamente ágeis. O modelo ágil dá maior ênfase à flexibilidade, à colaboração e à entrega contínua, e o modelo incremental não ágil pode ser mais estruturado em fases, com entrega de incrementos mais definidos.

O modelo RAD (Rapid Application Development) também pode ser considerado como um tipo de modelo incremental. Esse modelo prioriza a entrega rápida do produto de software, por meio do desenvolvimento de protótipos e iterações, geralmente utilizando equipes multifuncionais. O RAD, portanto, é uma abordagem que visa à entrega rápida de um sistema por meio do uso intensivo de protótipos, ou seja, versões preliminares e simplificadas do sistema, que são desenvolvidas durante a fase inicial do desenvolvimento.

Podemos dizer que o RAD é uma abordagem específica dentro da categoria mais ampla de modelos incrementais. Como o RAD é conhecido por suas entregas rápidas, com a ênfase na obtenção de um produto funcional em um curto período, ele é um modelo adequado para o projeto

descrito no enunciado da questão, já que a empresa solicitou extrema urgência na construção da aplicação.

A questão retrata uma situação que diz respeito, diretamente, a uma das áreas de conhecimento estipuladas no guia PMBOK (Project Management Body of Knowledge).

Áreas de conhecimento do PMBOK

O guia PMBOK é o principal documento de referência para os profissionais da gestão de projetos. Nele, são descritas boas práticas que devem ser adotadas por gestores.

O PMBOK está dividido em 10 áreas de conhecimento, visando a apresentar os processos críticos para a gestão de projetos. De forma resumida e simplificada, cada uma dessas áreas é apresentada, a seguir.

- Integração: o gerenciamento da integração do projeto envolve tomadas de decisão ligadas aos objetivos do projeto, assim como o processo de desenvolvimento e de execução do plano de gerenciamento. De forma resumida, essa área faz a coordenação dos diversos elementos do projeto.
- Escopo: o gerenciamento do escopo é relativo ao atendimento dos requisitos para o desenvolvimento e a finalização do projeto. Entre as responsabilidades dessa área, estão a coleta e a definição dos requisitos.
- Tempo: o gerenciamento do tempo é relativo a processos que asseguram a conclusão do projeto nos prazos estabelecidos.
- Custo: a gestão dos custos visa a garantir que o projeto será concluído dentro do orçamento aprovado.
- Qualidade: o gerenciamento da qualidade objetiva garantir que o projeto seja concluído em conformidade com os requisitos e com as especificações definidas em seu escopo.
- Recursos humanos: o gerenciamento de recursos humanos visa a utilizar de forma eficaz a força de trabalho das pessoas envolvidas no desenvolvimento do projeto.
- Comunicações: o gerenciamento de comunicações inclui um conjunto de processos que asseguram a geração, a coleta, a distribuição, o armazenamento e o controle das informações do projeto.
- Riscos: o gerenciamento de riscos envolve a definição, a análise e a resposta aos riscos do projeto. Ele antecipa futuras necessidades ou mudanças, permitindo a tomada de decisão com antecedência.
- Aquisições: o gerenciamento de aquisições procede determinando o que, quando e como comprar ou adquirir bens ou serviços externos à organização.
- Partes interessadas: o gerenciamento de partes interessadas consiste em processos necessários para identificar, planejar, gerenciar e controlar as partes interessadas de um projeto. Essa área é responsável por coordenar as interações entre os stakeholders.

Fonte: Material Instrucional Específico – Análise e Desenvolvimento de Sistemas – Tomo 14 – CQA/UNIP (introduções teóricas). Texto e figuras reproduzidos do material original.